DESARROLLO ORIENTADO A OBJETOS [DOPO]
2026-2
Ciclo 1

Materia: Desarrollo orientado a objetos
Grupo: 7

Integrantes

Santiago Murillo Ibañez
ID: 1000104973
Correo: santiago.murillo-i@mail.escuelaing.edu.co

Andres Steven Oyuela Mendez
ID: 1000109048
Correo: andres.oyuela-m@mail.escuelaing.edu.co

A continuación, se presenta una breve documentación sobre los atributos y métodos del **diseño de dominio** propuestos para esta primera entrega. Cabe destacar que, al tratarse de una propuesta generalizada, estos elementos no corresponden a la versión final del proyecto.

Match (Partida)

- **Atributos:**
    - estado: int - Estado actual de la partida (ej. no iniciada, en juego, pausada, finalizada).
    - isGameOver: boolean - Indica si la partida ha concluido definitivamente.
- **Métodos:**
    - startMatch() - Inicializa los componentes y comienza la partida.
    - updateLogic() - Actualiza la lógica del juego en cada ciclo o frame.
    - endMatch() - Finaliza la partida y gestiona el cierre del encuentro.
    - checkWinner() - Evalúa cuál jugador ha ganado la partida según las reglas.

ScoreBoard (Marcador)

- **Atributos:**
    - scorePlayer1: int - Puntuación o goles acumulados por el jugador 1.
    - scorePlayer2: int - Puntuación o goles acumulados por el jugador 2.
- **Métodos:**
    - addGoalPlayer1() - Incrementa en uno el marcador del jugador 1.
    - addGoalPlayer2() - Incrementa en uno el marcador del jugador 2.
    - getScore() - Devuelve el resultado actual del marcador.

Timer (Temporizador)

- **Atributos:**
    - timeRemaining: float - Tiempo restante de juego en segundos.
- **Métodos:**
    - startTimer() - Inicia el conteo regresivo del tiempo de juego.
    - stopTimer() - Detiene o pausa el temporizador.
    - isTimeUp(): boolean - Comprueba si el tiempo reglamentario ha finalizado.

Stadium (Estadio/Cancha)

- **Atributos:**
    - nombre: String - Nombre del estadio o escenario seleccionado.
    - izq: Goal - Referencia a la portería ubicada a la izquierda del campo.

    - der: Goal - Referencia a la portería ubicada a la derecha del campo.
    - friccionSuelo: float - Coeficiente de fricción que afecta el movimiento del balón y personajes en el suelo.
- **Métodos:**
    - getLimitesCancha() - Retorna los bordes o colisiones delimitadoras del escenario.
    - renderBackground() - Dibuja el fondo y elementos visuales del estadio.

Goal (Arco/Portería)

- **Atributos:**
    - idDueño: int - Identificador del jugador asignado a defender la portería.
    - coordenadasColision: Rectangulo - Área física que detecta cuando el balón entra a la portería.
- **Métodos:**
    - verificarIngresoBalon(balon: Ball): boolean - Determina si el balón cruzó el área de gol.

Result (Resultado)

- **Atributos:**
    - idGanador: int - Identificador del jugador triunfador del encuentro.
    - golesP1: int - Cantidad final de goles marcados por el jugador 1.
    - golesP2: int - Cantidad final de goles marcados por el jugador 2.
    - monedasGanadas: int - Monedas otorgadas como recompensa tras concluir la partida.
- **Métodos:**
    - calcularRecompensas() - Determina las recompensas en monedas según el desempeño.
    - mostrarPantallaFinal() - Muestra la interfaz gráfica con el resumen del resultado.

2. Actores y Físicas

Player (Jugador Físico)

- **Atributos:**
    - playerId: int - Identificador único del jugador físico en la partida.
    - energyBar: float - Nivel actual de energía para ejecutar tiros especiales.
    - controls: String[] - Mapeo de teclas o entradas asignadas a las acciones del jugador.
- **Métodos:**

    - move() - Procesa y aplica el movimiento horizontal del jugador.
    - jump() - Ejecuta la acción de salto aplicando una fuerza vertical.
    - head() - Realiza la acción de cabeceo hacia el balón.
    - kick() - Efectúa la acción de patada al balón.
    - chargeEnergy() - Incrementa progresivamente la barra de energía especial.
    - castSpecialShot() - Activa y ejecuta el tiro o habilidad especial cuando la energía está llena.

Character (Personaje/Avatar)

- **Atributos:**
    - name: String - Nombre identificador del personaje/avatar.
    - speed: float - Atributo base de velocidad de desplazamiento.
    - jumpForce: float - Atributo base de fuerza o altura de salto.
    - strength: float - Atributo base de potencia de tiro e impacto.
    - isStunned: boolean - Estado que indica si el personaje se encuentra aturdido o inmovilizado.
- **Métodos:**
    - applyStun(duration: float) - Aplica un efecto de aturdimiento por un tiempo determinado.
    - resetStats() - Restablece los atributos del personaje a sus valores base iniciales.

Ball (Balón - Clase Abstracta)

- **Atributos:**
    - posX: float - Posición horizontal actual del balón en el espacio 2D.
    - posY: float - Posición vertical actual del balón en el espacio 2D.
    - velocityX: float - Componente horizontal de la velocidad del balón.
    - velocityY: float - Componente vertical de la velocidad del balón.
- **Métodos:**
    - applyForce(forceX: float, forceY: float) - Altera la velocidad del balón mediante un vector de fuerza.
    - checkCollision(stadiumBounds) - Detecta y resuelve colisiones contra paredes o suelo.
    - resetPosition() - Devuelve el balón a la posición central de saque.

Soccer (Balón Tradicional - Hereda de Ball)

- **Atributos:**

    - reboteEstandar: float - Coeficiente de restitución por defecto para balones estándar.
    - friccionAire: float - Resistencia del aire estándar que desacelera el balón en trayectoria.
- **Métodos Sobrescritos:**
    - applyForce() - Aplica física de movimiento regular según las propiedades del balón convencional.

Basket (Balón de Baloncesto - Hereda de Ball)

- **Atributos:**
    - multiplicadorRebote: float - Coeficiente que incrementa la elasticidad del rebote.
    - pesoExtra: float - Masa añadida que modifica el efecto de la gravedad en el balón.
- **Métodos Sobrescritos:**
    - checkCollision() - Calcula la respuesta física del rebote adaptada al comportamiento de baloncesto.

Bomb (Balón Bomba - Hereda de Ball)

- **Atributos:**
    - tiempoParaExplosion: float - Duración de la cuenta regresiva antes de detonar.
    - radioDeImpacto: float - Distancia de alcance del efecto explosivo al detonar.
    - estaActivada: boolean - Indica si el temporizador de la bomba está en marcha.
- **Métodos:**
    - iniciarTemporizador() - Activa la cuenta regresiva para la detonación.
    - explotar() - Genera la detonación afectando a los elementos dentro del área de impacto.
    - aturdirJugadoresCercanos(p1: Character, p2: Character) - Inmoviliza temporalmente a los personajes que se encuentren dentro del radio de la bomba.

3. Sistema de Habilidades

Skill (Habilidad Especial - Clase Abstracta)

- **Atributos:**
    - energyCost: float - Cantidad de energía requerida para ejecutar la habilidad.
    - effectDuration: float - Tiempo durante el cual se mantendrá activo el efecto.
- **Métodos:**
    - activateEffect(ball: Ball, opponent: Character) - Ejecuta el efecto especial sobre el balón o el rival.

FireBall / Freeze / Bigger / Teleport (Habilidades Específicas)

- **Métodos Sobrescritos:**
    - activateEffect() - Aplica la mecánica propia de la habilidad (tiro de fuego, congelar, agrandar personaje o teletransporte).

4. Modos de Juego (Patrón Strategy)

GameMode (Modo de Juego - Clase Abstracta)

- **Atributos:**
    - nombreModo: String - Nombre identificador de la modalidad de juego.
    - multiplicadorRecompensas: float - Factor por el cual se multiplican las recompensas al ganar en este modo.
- **Métodos:**
    - iniciarReglas(partida: Match) - Configura los parámetros iniciales específicos del modo.
    - evaluarCondicionVictoria(marcador: ScoreBoard, timer: Timer): boolean - Determina si se cumplieron las condiciones para finalizar la partida.

Casual (Hereda de GameMode)

- **Métodos Sobrescritos:**
    - evaluarCondicionVictoria() - Evalúa la victoria por mayor puntuación al término del tiempo estándar.

Survival (Hereda de GameMode)

- **Atributos:**
    - vidasRestantes: int - Cantidad de oportunidades acumuladas antes de perder la sesión.
    - rondaActual: int - Número de nivel o rival consecutivo superado.
- **Métodos:**
    - aumentarDificultadRival() - Incrementa las estadísticas o la IA del oponente para el próximo nivel.
    - restarVida() - Reduce en una unidad el contador de vidas tras sufrir una derrota.

Tournament (Hereda de GameMode)

- **Atributos:**
    - faseActual: String - Nombre de la ronda eliminatoria actual (ej. Octavos, Cuartos, Final).
- **Métodos:**

- avanzarLlave() - Hace progresar al jugador a la siguiente etapa del torneo.
- generarSiguienteRival() - Selecciona y asigna el próximo oponente según el cuadro de competición.

Surrender (Interfaz)

- **Métodos:**
  - rendirse(idJugador: int) - Permite a un jugador abandonar prematuramente la partida.
  - aplicarPenalizacion() - Aplica un castigo o deducción por abandono voluntario.

5. Metajuego: Perfil, Inventario y Tienda

Profile (Perfil de Usuario)

- **Atributos:**
  - username: String - Nombre de usuario del jugador registrado.
  - totalCoins: int - Cantidad total de monedas acumuladas en la cuenta.
  - matchesWon: int - Histórico total de partidas ganadas.
- **Métodos:**
  - addCoins(amount) - Suma una cantidad de monedas al saldo del perfil.
  - equipCharacter(character: Character) - Selecciona el personaje principal activo para las partidas.

Inventory (Inventario)

- **Atributos:**
  - personajesDesbloqueados: Character[] - Lista de personajes adquiridos por el usuario.
  - guayosComprados: Guayos[] - Colección de objetos de calzado comprados.
  - personajeEquipado: Character - Personaje actualmente seleccionado para competir.
  - guayosEquipados: Guayos - Calzado de equipamiento activo en este momento.
- **Métodos:**
  - agregarItem(item: Purchasable) - Añade un nuevo elemento comprado a la colección del inventario.
  - equiparPersonaje(c: Character) - Asigna el personaje escogido a la ranura de activo.
  - equiparGuayos(g: Guayos) - Asigna el calzado escogido a la ranura de activo.

Shop (Tienda)

- *(Gestiona las transacciones entre el Profile y los items Purchasable).*

Purchasable (Interfaz)

- **Métodos:**
    - getPrecio(): int - Retorna el costo en monedas requerido para adquirir el elemento.
    - isComprado(): boolean - Verifica si el objeto ya ha sido desbloqueado por el usuario.
    - desbloquearItem(): void - Marca el objeto como adquirido de forma permanente.

Guayos (Zapatos - Implementa Purchasable)

- **Atributos:**
    - id: int - Identificador único del modelo de calzado.
    - nombre: String - Nombre comercial o descriptivo del calzado.
    - bonoVelocidad: float - Incremento otorgado a la velocidad del personaje.
    - bonoSalto: float - Incremento otorgado a la fuerza de salto del personaje.
    - bonoFuerza: float - Incremento otorgado a la potencia de golpe del personaje.
- **Métodos:**
    - aplicarMejoras(personaje: Character) - Modifica los parámetros base del personaje sumando las bonificaciones.
    - getPrecio(): int - Devuelve el valor monetario del calzado. *(Implementado de la interfaz)*

A continuación, se presenta una breve documentación sobre los atributos y métodos del **diseño de presentación** propuestos para esta primera entrega. Cabe destacar que, al tratarse de una propuesta generalizada, estos elementos no corresponden a la versión final del proyecto.

HeadSoccerGUI (Ventana Principal - Hereda de JFrame)

- **Atributos:**
    - domain: HeadSoccer - Referencia a la clase principal del dominio que conecta la interfaz con la lógica del juego.
    - cardLayout: CardLayout - Gestor de diseño que permite alternar entre las diferentes pantallas de la ventana.
    - menuPanel: MenuPanel - Panel correspondiente a la vista del menú principal y selección de modos.

    - matchPanel: MatchPanel - Panel correspondiente al escenario 2D donde transcurre la partida.
    - shopPanel: ShopPanel - Panel correspondiente a la interfaz gráfica de la tienda.
    - inventoryPanel: InventoryPanel - Panel correspondiente a la vista del inventario y equipamiento.
- **Métodos:**
    - prepareElements() - Inicializa la ventana, define sus dimensiones y construye los paneles secundarios.
    - prepareActions() - Configura los escuchadores de eventos para los botones de navegación y el cierre de la ventana.
    - showScreen(screenName: String) - Cambia la vista activa en la ventana principal.
    - refresh() - Solicita información actualizada al dominio y repinta los componentes visuales activos.
    - main(args: String[]) - Punto de entrada de la aplicación que instancia y hace visible la interfaz gráfica.

2. Vistas de Navegación y Metajuego

MenuPanel (Menú Principal - Hereda de JPanel)

- **Atributos:**
    - lblProfileInfo: JLabel - Etiqueta superior que muestra el nombre del perfil, monedas acumuladas y victorias.
    - btnCasual: JButton - Botón para iniciar una partida en modo Casual.
    - btnSurvival: JButton - Botón para iniciar una partida en modo Supervivencia.
    - btnTournament: JButton - Botón para iniciar una partida en modo Torneo.
    - btnShop: JButton - Botón de acceso directo a la pantalla de la tienda.
    - btnInventory: JButton - Botón de acceso directo a la pantalla del inventario.
- **Métodos:**
    - prepareElements() - Organiza visualmente el título del juego, la barra de perfil y los botones de selección.
    - prepareActions() - Vincula cada botón con el cambio de pantalla o la inicialización del modo seleccionado.
    - updateProfileHeader(username: String, coins: int, wins: int) - Actualiza en pantalla los datos del perfil del jugador.

InventoryPanel (Vista de Inventario y Pre-Partida - Hereda de JPanel)

- **Atributos:**
    - characterPreview: JLabel - Imagen del personaje actualmente seleccionado.
    - barSpeed: JProgressBar - Barra visual que representa la estadística de velocidad del personaje.
    - barJump: JProgressBar - Barra visual que representa la estadística de salto del personaje.
    - barStrength: JProgressBar - Barra visual que representa la estadística de fuerza del personaje.
    - cmbGuayos: JComboBox - Selector de los guayos adquiridos para equipar al personaje.
    - cmbStadium: JComboBox - Selector del estadio o cancha para la partida.
    - cmbBall: JComboBox - Selector del tipo de balón (Soccer, Basket, Bomb).
- Métodos:
    - prepareElements() - Construye las listas desplegables, la vista previa del avatar y las barras de estadísticas.
    - prepareActions() - Registra los eventos al cambiar de personaje, guayos, cancha o balón.
    - updateStatsView() - Recalcula y refresca las barras de progreso sumando las estadísticas base y los bonos de los guayos.

ShopPanel (Vista de Tienda - Hereda de JPanel)

- **Atributos:**
    - itemsGrid: JPanel - Contenedor en cuadrícula donde se muestran las tarjetas de personajes y guayos en venta..
    - lblAvailableCoins: JLabel - Indicador visual del saldo de monedas disponibles.
    - btnBuy: JButton - Botón para ejecutar la compra del ítem seleccionado.
    - btnBack: JButton - Botón para regresar al menú principal.
- **Métodos:**
    - prepareElements() - Carga el catálogo visual de elementos comprables con sus respectivos precios.
    - prepareActions() - Asigna la acción de compra y valida visualmente si el ítem ya fue adquirido.

- refreshCatalog() - Actualiza el estado de los botones a "Adquirido" y descuenta las monedas en pantalla tras una compra.

3. Vista de Partida en Tiempo Real y Resultados

MatchPanel (Lienzo de Juego 2D - Hereda de JPanel)

- **Atributos:**
  - renderTimer: Timer - Reloj gráfico encargado de refrescar el lienzo constantemente durante la partida.
  - lblScoreBoard: JLabel - Componente que muestra los goles del Jugador 1 y Jugador 2.
  - lblTimer: JLabel - Etiqueta que muestra la cuenta regresiva del tiempo restante de juego
  - energyBarP1: JProgressBar - Barra gráfica de carga de energía para el tiro especial del Jugador 1.
  - energyBarP2: JProgressBar - Barra gráfica de carga de energía para el tiro especial del Jugador 2.
  - btnPause: JButton - Botón para pausar el encuentro.
  - btnSurrender: JButton - Botón para rendirse cuando el modo de juego lo permite.
  - keyListener: KeyAdapter - Capturador de teclado para procesar simultáneamente los controles de ambos jugadores.
- **Métodos:**
  - prepareElements() - Ubica el marcador, el reloj y las barras de energía en el panel superior.
  - prepareActions() - Configura el escuchador de teclado y las acciones de los botones.
  - paintComponent(g: Graphics) - Dibuja en cada ciclo el fondo del estadio, las dos porterías, los personajes, el balón y los efectos visuales.
  - updateHUD() - Sincroniza en pantalla los goles, el tiempo restante y las barras de energía con la lógica del juego.

ResultDialog (Ventana Emergente de Resultado - Hereda de JDialog)

- **Atributos:**
  - lblWinner: JLabel - Texto principal que anuncia al jugador ganador de la partida.
  - lblFinalScore: JLabel - Resumen con el marcador final de goles.

    - lblCoinsEarned: JLabel - Cantidad de monedas otorgadas como recompensa tras el encuentro.
    - btnRematch: JButton - Botón para reiniciar inmediatamente una nueva partida.
    - btnMainMenu: JButton - Botón para regresar al menú principal.
- BallSprite
    - prepareElements() - Diseña el cuadro de diálogo con los textos de resumen y botones de acción.
    - prepareActions() - Configura las acciones para jugar revancha o volver al menú.
    - displayResult(winner: int, goalsP1: int, goalsP2: int, coins: int) - Rellena los campos con los datos del resultado final y muestra la ventana.

N**omenclatura húngara**

- **lbl (Label / Etiqueta):** Derivado de JLabel. Designa componentes estáticos destinados a mostrar información textual o gráfica no editable por el usuario.
- **btn (Button / Botón):** Derivado de JButton. Identifica elementos interactivos que activan un evento o función al ser presionados.
- **pnl (Panel / Contenedor):** Derivado de JPanel. Corresponde a contenedores que agrupan otros componentes visuales para estructurar la vista.
- **txt (Text / Campo de Texto):** Derivado de JTextField o JTextArea. Se aplica en campos de entrada donde el usuario ingresa texto de forma directa.

Se cuenta con una idea estructural del pq se sugiere utilizar el CharacterSprite y el BallSprite

- Esto con el fin de que los prefijos hacen que el código de presentación sea legible y profesional, y los *Sprites* mantienen tu panel de juego limpio, delegando la responsabilidad gráfica a los objetos que realmente representan en la pantalla y así cada sprite se encarga de su propia animación.